HACKATHON SUBMISSION · TECHNICAL REPORT

Godavari Crowd Monitor

Real-time crowd counting, zone congestion and capacity alerting for mass gatherings — one service, no cloud, measured on real footage.

32 /s

inferences per second on a consumer RTX 3050, four live cameras at 8 FPS each

102 FPS

YOLOv8x inference on an RTX PRO 6000 (960 px), 36 FPS end to end

1 → 135

people found on one dense frame once resolution is escalated — shown as ≥ 135, a lower bound, never a confident guess

15 clips

real Kumbh, Hajj and platform crowds used for evaluation — including where it fails

01 · THE PITCH

Counting people is easy. Knowing a zone is about to fail is not.

The problem

- **Control rooms watch, they cannot count.** One operator cannot judge 40 feeds, so congestion builds in the one zone nobody is looking at.
- **Counts are not decisions.** "312 people detected" means nothing until you know how many that gate can hold. Occupancy is the number an operator acts on.
- **Alarms without discipline get switched off.** A detector that fires on every flickering frame trains staff to ignore it — the worst outcome for a safety tool.
- **Analytics suites are heavy.** A broker, a metrics database, a GPU cluster and a licence negotiation stand between a volunteer event and any monitoring at all.

What Godavari does

- Turns any camera — uploaded clip, bundled sample, RTSP CCTV stream or webcam — into a **zone-aware occupancy sensor**.
- Detects people, gives each a persistent identity, and measures **occupancy % of a declared capacity** per polygon zone, plus relative density and flow direction.
- Raises **one warning** on the rising edge and a **capacity alert only after sustained overload**, with cooldown, chime, SQLite history and CSV export.
- Ships as **one container**: no broker, no database server, no cloud account.

Why it stands apart

Occupancy, not a raw count

Each zone has a capacity. The dashboard shows *count / capacity*, occupancy %, density and a colour level — the same mental model as a venue's own capacity limit.

Alerting with hysteresis

Warning at 75 % on the rising edge; OVERCROWDED only after 5 s at ≥ 100 %; repeats no more than once a minute. Signal, not noise — and every alert is persisted.

Runs where the budget is

3.6 inferences/s on a CPU-only laptop for demos and small events; 32/s on a second-hand GPU; 100+ FPS on server silicon. Same image, one build arg apart.

Honest engineering

Every number in this report was measured on this build, including the uncomfortable ones: box detectors under-count packed crowds. Where the picture is too small to resolve people, the dashboard shows **≥ 135** and says “lower bound”, never a confident wrong number.

Status: a working system, not a mock. Camera add/remove, polygon zones, occupancy, alerts, history, MJPEG streaming, heatmap, flow and wall view all run today — verified in a browser, in Docker on CPU, and in Docker on a GPU.

02 · HOW IT WORKS

One process, one model, one honest bottleneck

Ingest
upload · sample clip · `rtsp://` · webcam index; files loop, streams reconnect

Capture thread
per camera, downscale to `CROWD_MAX_FRAME_WIDTH`

Detect
YOLO26n (NMS-free) on CUDA or CPU, one shared model behind a lock

▼

Track
ByteTrack IDs — a person is not counted twice

Measure
foot point → polygon zones → count, occupancy %, density

Derive
flow vector + speed, 48×27 decaying heatmap

▼

Alert
sustain + cooldown → SQLite history, chime, CSV

Broadcast
snapshot every 0.6 s over WebSocket

Stream
annotated MJPEG at 12 FPS per camera

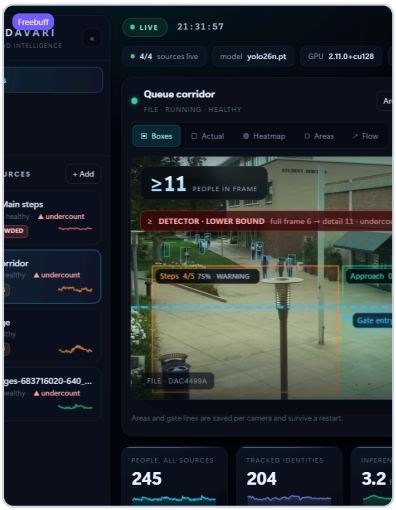
Four decisions that make it usable

- **One model, many cameras.** Memory stays flat as sources are added; each camera gets its own inference budget, and the lock makes queuing predictable instead of thrashing the GPU.
- **Foot point, not box overlap.** A person counts as inside a zone when their bounding-box foot point falls in the polygon — overlapping queues stop double-counting.
- **EMA smoothing (60/40).** Occupancy is computed from a smoothed count, so a frame of missed detections cannot flash a zone red.
- **State machine, not thresholds.** OK → BUSY → WARNING → OVERCROWDED, with sustain timers and cooldown; the alarm reflects a condition, not a frame.

What the operator sees

- **One camera, five views** — boxes, the clean picture, heatmap, isolated areas and flow — plus an **all-views mosaic** of the same camera. Keys **1–5**, mosaic on **0**; click any tile to open it full size.
- **Areas drawn on the video**: new area → click corners → name, capacity, save. Stored per camera with a version, so the boundary survives a restart.
- **Gate lines** across an entrance count entries and exits in both directions.
- **Inspector** with three tabs: alerts and acknowledgement, the area/gate editor, and analytics trust per camera.
- **Trend** with alert markers — the run-up to a breach, not only the breach.
- **Wall view** of every camera as a tile, for a control-room display.

Dashboard stack	**vanilla JS + WebSocket (no framework)**
Transport	**MJPEG (annotated and raw) + JSON snapshots**



Live dashboard, rebuilt around one camera.
Collapsible sidebar with sources, engine and insights; view chips above the stage (**all views · boxes · actual · heatmap · areas · flow**); HUD with **≥ 11** people and the lower-bound badge; areas and gate line drawn on the video; KPIs and the occupancy trend below.

The same camera can be read five ways without losing the measurement, and the all-views mosaic shows every view at once — clicking a tile opens it full size.

Tech stack: small, current, and pinned for a reason

Layer	Technology	Version	Why this one
Language	Python	3.11 / 3.12	Native home of the vision and ML ecosystem; one language from ingest to API.
Detection	Ultralytics YOLO26n (YOLO11n / YOLOv8n fallback)	8.4.120	NMS-free, strong on small/dense objects, ~10 ms on GPU; nano variant keeps CPU viable.
Tracking	ByteTrack via `supervision`	0.18.0	Identity persistence for distinct-people counts; pinned because ≥0.24 removed `sv.ByteTrack`.
Vision	OpenCV	5.0.0.93	Decode, resize, annotate, JPEG encode. Standard wheel + `libgl1`, because the 5.0 headless wheel ships no `cv2`.
Numerics	PyTorch (+ torchvision)	2.11 CUDA 12.8 / 2.15 CPU	CUDA 12.8 is the first build with Blackwell (sm_120) kernels; cu121 silently fails there.
API	FastAPI + Uvicorn	0.141.1 / 0.52.4	Async REST, WebSocket and streaming in one process; typed models, zero boilerplate.
Frontend	Vanilla JS + CSS, Canvas sparklines	—	No build step, no framework, loads instantly from the same process.
Persistence	SQLite	stdlib	Alert history that survives restarts without operating a database server.
Packaging	Docker (slim-bookworm), Compose	CPU 0.67 GB / GPU 4.4 GB	Non-root uid 10001, healthcheck, weights baked in so startup needs no network.
Hardware path	CUDA 12.8 wheels, CPU fallback	sm_50 → sm_120	Same source runs on a laptop or a 96 GB server GPU; device chosen at start-up.

Deployment shapes verified

Environment	Build	Size
Windows dev	`tools/install.sh` → `.venv`	7.5 GB
Any host, CPU	`docker build -t godavari:cpu .`	0.67 GB
GPU host	`--build-arg TORCH_INDEX_URL=…/cu128`	4.4 GB
Ubuntu 24.04 VM	venv + cu128 wheels	7.5 GB

Dependencies are pinned in `requirements.txt`; every version above is the one actually installed and exercised, not a target.

Code layout

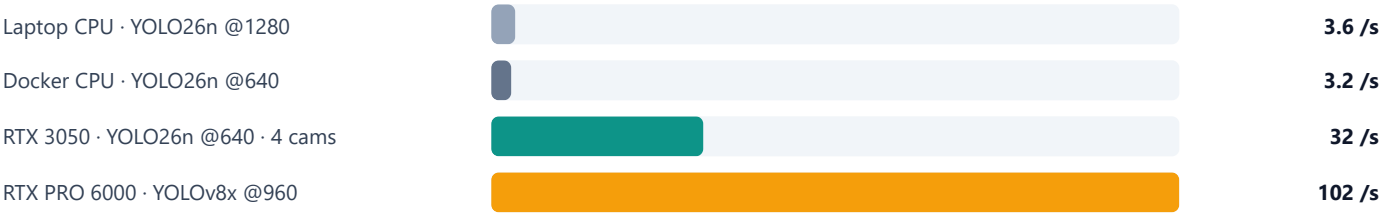
Path	Responsibility
`app/pipeline.py`	CameraWorker + registry (capture → detect → track → zones)
`app/detector.py`	Model manager, device selection, fallbacks
`app/zones.py`	Polygon geometry, occupancy, density, levels
`app/alerting.py`	Sustain + cooldown state machine
`app/heatmap.py`	Decaying 48×27 density grid
`app/store.py`	SQLite alert history, CSV export
`static/`	Dashboard (HTML/CSS/JS, no build)

04 · PERFORMANCE

Measured throughput across four environments

Environment	Detector	imgsz	Throughput	Reading
Development laptop, CPU-only torch	YOLO26n	1280	**3.6** inferences/s (1 camera); 1.8 FPS each with 2 cameras	Demo / small event
`godavari:cpu` container	YOLO26n	640	**3.2** FPS, one camera, healthy container in ~9 s	Cheap always-on VM
RTX 3050 8 GB laptop GPU	YOLO26n	640	**32** inferences/s — 4 cameras × 8.0 FPS, 1.8 GB VRAM, ~130 % CPU	4–6 cameras per box
RTX PRO 6000 (96 GB), Ubuntu 24.04	YOLOv8x	960	**102** FPS inference, **35.8** FPS end-to-end	Large headroom, heavier model

Aggregate inferences per second (higher is better; detectors differ, see note)



YOLOv8x is roughly an order of magnitude heavier than YOLO26n, so the server-GPU lead is larger than the bar lengths suggest. The two GPU numbers were produced by the same code path, only the weights and device changed.

How these were measured

- Real clips, real detector: 1280×720 H.264, 30 FPS source, 40–100 frames per run.
- **Inference FPS** = frames ÷ time spent inside `model.predict` only.
- **End-to-end FPS** = frames ÷ wall clock, so it includes decode, tracking, zone maths and annotation. Both are reported because quoting only the first is how benchmarks lie.
- GPU runs used `--gpus all` in Docker and a bare venv on the VM; both reported `cuda_available: true` before timing.
- Multi-camera figures come from four simultaneous sources with the 5 FPS cap lifted, so the number reflects compute, not the configured throttle.

Footprint and start-up

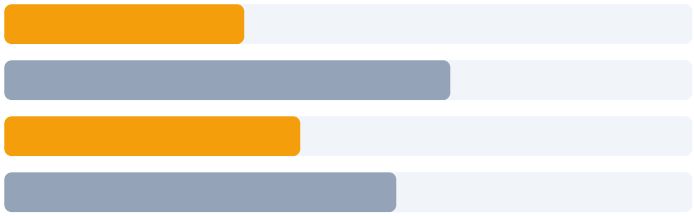
VRAM, 4 cameras, YOLO26n	**1.8 GB**
CPU image / GPU image	**0.67 / 4.4 GB**
Container to healthy	**~9 s**
Model weights baked in	**no network at boot**
Memory growth per camera	**thread + buffers only**
Database	**SQLite file, no server**

One shared model instance means adding the fifth camera does not add a second copy of the weights — the marginal cost is a capture thread and its frame buffers.

05 · ANALYSIS

Where the milliseconds actually go

The server GPU is not the constraint — the single-threaded decode and annotation path is. At 960 px, a YOLOv8x frame costs **9.8 ms** of inference inside a **28 ms** frame, so roughly two-thirds of the wall clock is spent outside the model:



What that means in practice

- **Per-camera cost is bounded by the lock.** N cameras share one model, so total throughput is capped by aggregate inference — not by camera count. That is why four cameras on a 3050 each land on 8.0 FPS instead of 5, 5, 5, 5.
- **MJPEG is the hidden CPU tax.** Every connected viewer costs JPEG encodes; at four cameras the 3050 test sat at ~130 % CPU while the GPU idled.
- **Scaling to 100 cameras is a deployment change, not a rewrite.** 100 cameras at 5 FPS is 500 inferences/s: batched inference (Triton), NVDEC decode and several processes. The single-process design is honest about this ceiling.
- **A laptop still works.** At 3.6 inferences/s the app serves one or two cameras at a real 3–5 FPS, which is enough for gate throughput counting.

Tuning knobs, in order of effect

Setting	Effect
`CROWD_IMGSZ` 640 → 1280	Better far-field recall, ~2× cost
`CROWD_MAX_FRAME_WIDTH`	Caps decode cost per frame
`CROWD_TARGET_INFER_FPS`	Per-camera budget; the main fairness dial
`CROWD_MODEL` n → s	Higher recall, ~3× compute
`CROWD_STREAM_FPS` / `JPEG_QUALITY`	Trades stream CPU for smoothness
`CROWD_WS_PERIOD`	Dashboard push rate vs. fan-out cost

Rule of thumb for an event: one GPU box per 4–6 cameras at 5 FPS with YOLO26n at 640 px — and a second box before you raise the resolution, because decode is what saturates first.

Scaling path, costed

Scale	Architecture	Change required
1–6 cameras	This application, one GPU container	None — deploy as-is
6–30 cameras	Several containers, one per GPU or per 5 cameras	Ops only; zones per camera already supported
30–100 cameras	MediaMTX fan-in, NVDEC decode, batched inference	New worker + Triton service (specified in `docs/PRODUCTION_ARCHITECTURE.md`)
100+ cameras	Sharded workers, TimescaleDB, server-side mosaic	Control plane and monitoring stack; the zone/alert model is unchanged

How good is it, on real crowds?

Fifteen licensed clips of actual mass gatherings — Maha Kumbh 2025, the Kaaba tawaf, Mumbai platforms — sampled at 3 s, 15 s and 30 s with YOLO26n at `imgsz=1280`. The results are useful precisely because they include the failures:

Clip	People detected	Reading
`kumbh_2025_river_walk` — packed one-way flow	23 / 29 / 19	**Reliable** dense but resolvable
`kumbh_2025_walk_4k` — walking tour	22 / 14 / 17	**Good** front row clear, back rows occluded
`tawaf_4k_walk` — circular mass at the Kaaba	15 / 20 / 17	**Partial** only the front rank resolves
`kumbh_2013_maha_shivratri` — elevated wide view	6 / 12 / 27	**Variable** tiny figures, swings with visibility
`station_rush_mumbai` — platform at 8 pm	8 / 9 / 7	**Under-counts** ~40 visible at the door
`kumbh_2025_crowd_walk` — night, backlit	6 / 10 / 2	**Degrades** lighting dominates
`railway_crowd_india` — overhead crush	1 / 3 / 3	**Fails** interlocked crowd is one texture
`hajj_stock_nocopyright` — aerial Haram	4	**Fails** tens of thousands visible, 4 found

Why the hard cases fail — and why that is expected

A bounding-box detector needs to see a whole body with edges to separate it from its neighbour. At platform-crush density there are no gaps, so people merge into texture. This is the documented behaviour of the entire box-detector family, not a defect in this code — and it is the single most important thing a buyer should know before trusting any such product's headline accuracy.

Where it is strong: resolvable, medium-density scenes — gate queues, walkways, plazas, station concourses — which is exactly where zone-capacity alerting is useful, because the trigger is a *trend* in occupancy, not a precise census.

Mitigations in place: 1280 px inference for far-field detail; smoothed counts so single-frame misses do not alarm; alerts that require sustained overload; capacities chosen per zone by the operator.

What is not claimed

- ✗ No labelled ground truth yet — so no mAP, precision or recall figure is claimed anywhere in this report.
- ✗ Density is *relative* (people per 10k px²), not metric people/m²; that needs a per-camera homography.
- ✓ Frozen, stale, dark, low-contrast and blurry cameras are detected and marked degraded, never reported as an empty scene.
- ✗ No cross-camera de-duplication; two views of one gate double-count.
- ✓ Next model addition: a head-point density estimator (P2PNet / DM-Count) as a cross-check — trust detection when sparse, density when dense.

Resolution is a limit you can pay to lift

Pass	People found	Cost
Full frame @ 640 (default)	0.3	10 ms
2×2 probe @ 640 — only if the default found ≤ 6	4.4	15 ms
3×3 detail @ 1280 — only if the probe disagrees	**135.4**	58 ms

768×432 Getty clip, several hundred people in frame. Two cheaper triggers were measured and rejected: edge density scored 0.42–0.89 on *every* clip (it reads texture, not people), and frame-to-frame motion was ~0.000 on standing crowds.

The number becomes a floor. A recoverable count is published as **≥ N** with `estimation_mode=detector_lower_bound`, capacity alerts are suppressed in that mode, and the operator gets one advisory per cooldown: "Dense crowd: 135 people detected is a lower bound, not a count — manual observation required."

07 · ENGINEERING QUALITY

What was actually verified

Behaviour

- ✓ Camera add → live count, in the browser and via API.
- ✓ Polygon zones drawn, saved, reloaded and enforced.
- ✓ Zone state machine reaches OVERCROWDED and alerts fire once per cooldown; history survives a restart; CSV export works.
- ✓ MJPEG stream and snapshot JPEG served for every source.
- ✓ Data-loss regression fixed: saved cameras load before model warm-up, so an early edit can no longer wipe `cameras.json`.
- ✓ Path traversal on sample names rejected with HTTP 400.
- ✓ Headless UI check reports no console problems.
- ✓ Detail escalation verified on a live camera: the same dense clip that reported **1** person reports **101–115** at `tier=detail` (≈70 ms/frame), and the badge reads **≥ N · LOWER BOUND**.
- ✓ Escalation is sticky per camera with hysteresis — a packed scene cannot alternate between "7" and "≥ 146" on consecutive frames. Verified by polling the API 8 times through a dense clip.
- ✓ Areas and gate lines drawn in the browser, written to `cameras.json`, and still present after a full service restart; every geometry change bumps `zones_version` and writes an audit row.
- ✓ Five camera views and the all-views mosaic: each tile's canvas hash differs, the raw stream serves 0 box-coloured pixels against ~255 for the annotated one, and a tile click returns to the single view and stops the extra stream.
- ✓ Alert acknowledgement from the UI records `ack_by=operator` with an audit entry.
- ✓ 30 unit tests covering router decisions, detail escalation, tiling, zones, alerts, health, calibration and the API.

Deployment

- ✓ CPU image built, healthy in ~9 s, camera at 3.2 FPS, zones applied, snapshot served.
- ✓ State persistence: `docker restart` kept the camera and its two zones (volume-backed).
- ✓ GPU image on an RTX 3050: `cuda: true`, four cameras, 32 inferences/s.
- ✓ Second GPU host (RTX PRO 6000, Blackwell) installed from scratch: sm_120 kernels confirmed with a real matmul, 102 FPS with YOLOv8x.
- ✓ Both compose files validate with `docker compose config`.

Security and operating posture

- Binds to **127.0.0.1** by default; publishing is a deliberate act.
- **No authentication by design** — the documented deployment puts TLS and a password in front (Caddy/nginx examples provided). This is stated in the README, not hidden.
- Container runs as **non-root uid 10001**; uploaded filenames are sanitised; sample paths are resolved and confined.
- No secrets, no telemetry, no outbound calls at runtime — weights are baked into the image.
- Healthcheck polls `/api/system` with a 120 s start period for cold model load; single-process design is documented because multiple workers would split the in-memory registry.

Licensing — read this before a public demo

Ultralytics YOLO is **AGPL-3.0**. Serving it as a network service to third parties can oblige you to publish corresponding source; internal use is normally fine, and a commercial deployment should buy a licence or swap the detector. Of the 15 test clips, only four are Creative Commons (one non-commercial) — the YouTube clips are *evaluation only* and must be replaced before any public demo.

Known limits, stated up front

Relative density	Needs homography for people/m²
Very dense crowds	Escalation recovers the resolvable people; the result is a lower bound, not a census. A head-point density model is still the only honest way to count an interlocked crowd.
Cost of detail	3×3 @1280 on a dense camera roughly halves its frame rate (≈8 → 4 FPS), which is why it is gated behind two cheap checks
Dead camera	Detected and marked degraded (stale/frozen/dark); those states never clear an alert
Delivery	Dashboard, CSV/PDF reports and signed webhooks with retries; no SMS or email sink yet
Scale-out	Single process; sharded design specified, not built

Pitch, demo and judging map

One-liner

Godavari turns any camera into a zone-aware crowd sensor: it measures occupancy against the capacity you declare, and shouts only when a zone stays over that line.

Thirty-second pitch

Every large gathering is still managed by eyeballs on a wall of screens. Counts do not help: an operator needs to know a zone is *over capacity*, and needs to know it before the crush, not after. Godavari adds polygon zones with real capacities to ordinary cameras, smooths the counts so they do not flicker, and raises one alert when a zone stays full — then writes it to a history you can export. It runs on a laptop CPU today, on a single GPU for four to six cameras, and ships as one 670 MB container with no cloud dependency.

Two-minute demo script

#	Action	What the judge sees
1	**+ Add** → *Pilgrim* → pack walk to the river	Live annotated feed with counts within seconds; status goes LIVE
2	**Draw zone** → gate polygon → capacity 8 → **Finish**	Zone chip appears with count/capacity and occupancy %, colour-coded
3	Toggle **H** then **Z**, then **F**	Heatmap of compression, zone polygons, flow arrow with speed
4	Lower the capacity below the current count and wait 5 s	WARNING on the rising edge → OVERCROWDED, chime, alert in the stream
5	Reload the page; then **Export CSV**	Alerts persist across restarts and leave the app as data

Judging-criteria map

Criterion	Evidence in this build	Where to look
Impact	Zone occupancy against declared capacity is the number a control room acts on; alerts are rate-limited so they stay trusted.	§02 zones & alerts
Technical depth	Detection + tracking + polygon geometry + EMA smoothing + hysteresis state machine + WebSocket fan-out in one process; CUDA/CPU parity; measured bottleneck analysis.	§03, §05, §06
Execution	Working UI, REST API, MJPEG, SQLite history, Docker CPU and GPU images, deployment guide, evaluated on 15 real clips.	§07
Originality	Occupancy-first model with alert discipline, rather than another detection demo; states its own failure modes.	§01, §06
Feasibility	Runs on a laptop today; scaling path costed by camera count with named technologies.	§05, §06
Integrity	No fabricated accuracy: no mAP claimed without labels; AGPL and footage licences disclosed.	§06, §08

Demo assets

- Dashboard screenshot (this document), plus live app on loopback.
- 15-clip pilgrim library, 45 s each, instant start.
- Container images: CPU 0.67 GB, GPU 4.4 GB.

Team & links (fill in)

- Team: name · members · roles
- Repository: URL · Demo video: URL
- Contact: email

Roadmap and reference

Next 90 days

Item	Why it matters
Density cross-check (P2PNet/DM-Count)	Turns the packed-crowd lower bound into an estimate with a calibrated error
Per-camera homography → people/m²	Turns relative density into safety bands
Labelled gate clip → first real mAP	Replace qualitative claims with metrics
Auth + role-scoped API keys	Multi-operator deployments
TimescaleDB + SSE fan-out	Retention, aggregates, restarts without dropping dashboards
MediaMTX + NVDEC + Triton	The 100-camera step; keeps the current path as the dev/CPU path

Effort is deliberately not quoted per item: each is a week-scale task for one engineer, and the first two unlock the others.

Appendix B · Key settings

`CROWD_MODEL`	yolo26n.pt
`CROWD_CONF`	0.30
`CROWD_IMGSZ`	640
`CROWD_DETAIL_PASS`	on
`CROWD_DETAIL_PROBE_BELOW`	6
`CROWD_DETAIL_IMGSZ / GRID`	1280 / 3
`CROWD_WARN_PCT`	75
`CROWD_ALERT_SUSTAIN / COOLDOWN`	5 s / 60 s
`HOST / PORT`	127.0.0.1 / 8000

Appendix A · HTTP API

`GET /api/cameras`	All cameras + live metrics
`POST /api/cameras`	Add file / sample / RTSP source
`GET /api/cameras/{id}`	One camera
`DELETE /api/cameras/{id}`	Stop and remove
`PUT /api/cameras/{id}/zones`	Replace polygon areas (bumps version)
`PUT /api/cameras/{id}/count-lines`	Gate lines (entry/exit counts)
`PUT /api/cameras/{id}/display`	Bake detection boxes or not
`GET /api/cameras/{id}/stream`	Annotated MJPEG; `?raw=1` for the clean frame
`GET /api/cameras/{id}/snapshot`	Single annotated JPEG
`GET /api/samples`	Sample library
`GET /api/alerts`	Alert history
`POST /api/alerts/{id}/ack`	Acknowledge, with audit row
`GET /api/alerts/export`	CSV export
`GET /api/analytics`	Forecasts, growth, queues, transitions
`GET /api/reports/daily.pdf`	Daily PDF report
`GET /api/audit`	Operator and system audit trail
`GET /api/models`	Registry, licence, swap
`GET /api/health · /metrics`	Scheduler and Prometheus metrics
`GET /api/system`	Model, device, uptime
`WS /ws`	Snapshot push every 0.6 s

Reproduce: `bash tools/install.sh` → `bash tools/start_dev.sh`, add a sample camera, read `/api/system`. Containers per `docs/DEPLOYMENT.md`; clip metrics with `tools/fetch_footage.py` and `tools/bench_models.py`.